

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Output Format

- For each process, display whether it is allocated or not.
- If allocated, display the block number, block size, and internal fragmentation.
- If not allocated, display that the process could not be allocated.

Example 1

Sample Input 1

```
5
100 500 200 300 600
4
212 417 112 426
```

Sample Output 1

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
```

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n;
5
6     // Input for memory blocks
7     printf("Enter number of memory blocks:\n");
8     scanf("%d", &m);
9
10    // Using Variable Length Arrays (C99 standard)
11    int blockSize[m];
12    int originalBlockSize[m];
13
14    printf("Enter size of each block:\n");
15    for (int i = 0; i < m; i++) {
16        scanf("%d", &blockSize[i]);
17        originalBlockSize[i] = blockSize[i]; // Keep original sizes for the final output
18    }
19
20    // Input for processes
21    printf("Enter number of processes:\n");
22    scanf("%d", &n);
23
24    int processSize[n];
```

Test Cases 2

Run Sample Cases

Run All Cases

Select Language: C (GCC 9.2.0)

```
25 printf("Enter size of each process:\n");
26 for (int i = 0; i < n; i++) {
27     scanf("%d", &processSize[i]);
28 }
29
30 // Best Fit Allocation Logic
31 for (int i = 0; i < n; i++) {
32     int best_idx = -1; // Initialize to -1 to track if we found a fit
33
34     for (int j = 0; j < m; j++) {
35         // Check if block can fit the process
36         if (blockSize[j] >= processSize[i]) {
37             // If it's the first block that fits, or if it's a tighter fit than the current best
38             if (best_idx == -1 || blockSize[j] < blockSize[best_idx]) {
39                 best_idx = j;
40             }
41         }
42     }
43
44     // If a suitable block was found
45     if (best_idx != -1) {
46         blockSize[best_idx] -= processSize[i];
47     }
48 }
49
```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

Sample Output 1

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with
Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with
Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with
Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with
Fragment 174
```

Example 2

Sample Input 2

```
3
100 200 300
```

Select Language: C (GCC 9.2.0)

```
39     if (best_idx == -1 || blockSize[j] < blockSize[best_idx]) {
40         best_idx = j;
41     }
42 }
43
44 // If a suitable block was found
45 if (best_idx != -1) {
46     blockSize[best_idx] -= processSize[i];
47
48     int fragment = blockSize[best_idx];
49
50     printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
51           i + 1, processSize[i], best_idx + 1, originalBlockSize[best_idx], fragment);
52 } else {
53     printf("Process %d of size %d is not allocated\n", i + 1, processSize[i]);
54 }
55 }
56
57 return 0;
58 }
59
60
61
62 }
```

Test Cases 2

Run Sample Cases

Run All Cases